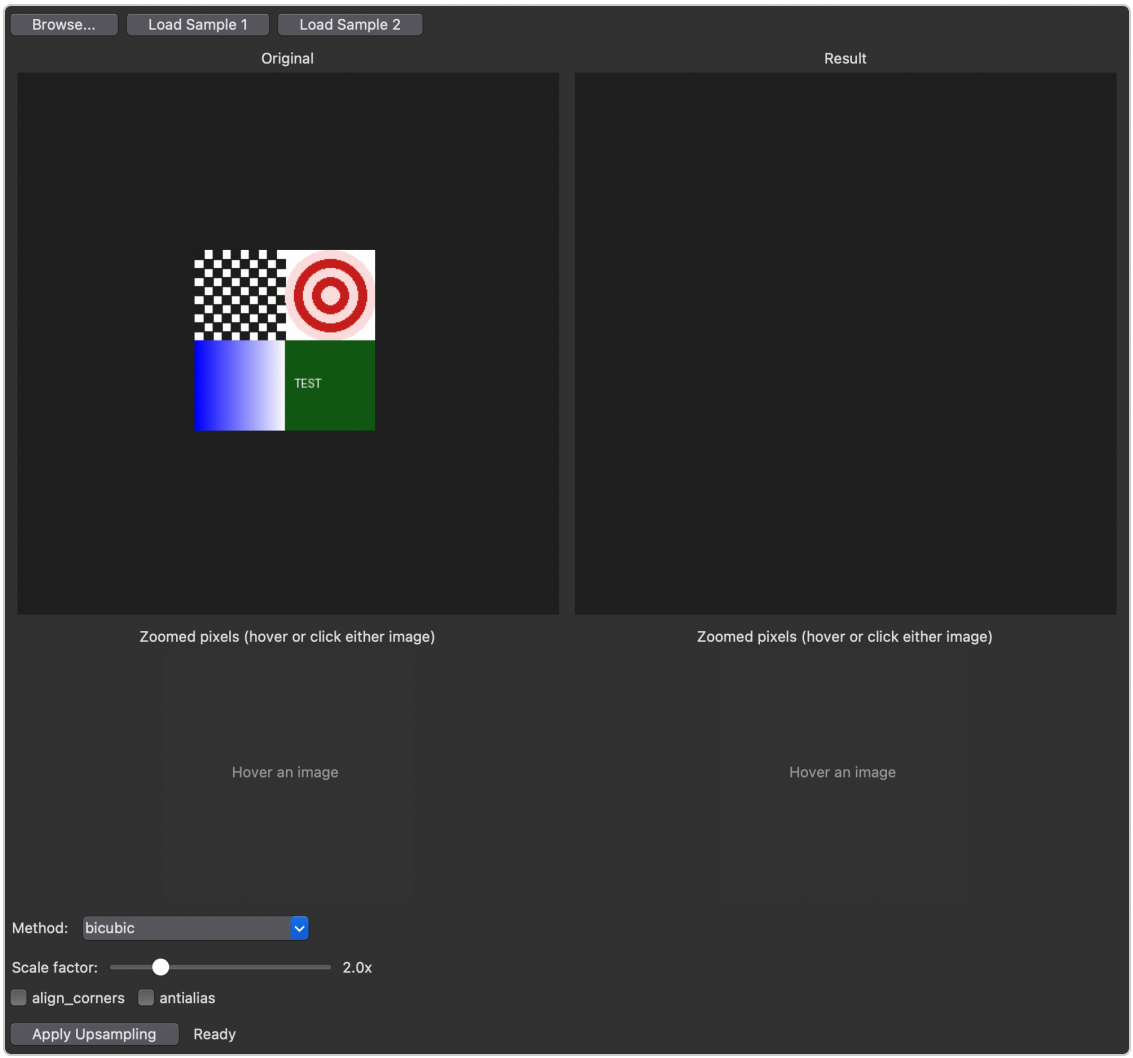
PyTorch Upsampling Comparison

Theory and Implementation



A desktop GUI (Tkinter + PyTorch) for visually comparing classic image-upsampling algorithms against a pretrained deep-learning super-resolution model.

Repository: github.com/pablo-figueroa-uniandes/pytorch-upsampling-gui

Contents

1. Overview
2. Theory of Image Upsampling
    a. The general problem
    b. Nearest-neighbor interpolation
    c. Bilinear interpolation
    d. Bicubic interpolation and overshoot
    e. Area interpolation
    f. `align_corners` and `antialias`
    g. Deep-learning super-resolution (EDSR, MSRN, DRLN)
    h. Side-by-side comparison
3. Application Walkthrough
4. Code Architecture
5. Module-by-Module Code Listing
6. Design Decisions Summary

1. Overview

This project is a small, self-contained desktop application for exploring how different image-upsampling techniques work and how they compare visually. It is built with **Tkinter** (Python's standard-library GUI toolkit) for the interface and **PyTorch** for every actual pixel computation. The user loads an image (a bundled synthetic sample or any file from disk), picks an upsampling method from a dropdown, adjusts that method's parameters, and clicks *Apply* to see the result next to the original. A pixel-level magnifying glass lets the user inspect the same physical region of both images at once, at the individual-pixel level.

Two families of methods are available:

- **Classic interpolation** — nearest, bilinear, bicubic, and area, all implemented by `torch.nn.functional.interpolate`. These are purely mathematical resampling procedures: every output pixel is some combination of pixels already present in the input.
- **Pretrained super-resolution** — a choice of three architectures, EDSR, MSRN, and DRLN, each loaded via the `super-image` package with weights downloaded from the Hugging Face Hub. These are convolutional neural networks trained to synthesize plausible new detail rather than just recombine existing pixels.

2. Theory of Image Upsampling

2.1 The general problem

An image is a grid of samples of some underlying continuous scene, taken at a fixed resolution. Upsampling asks: given only those samples, what value should we assign to a *new* grid point that falls *between* the original samples? Every method in this app answers that question differently, and the differences become visually obvious once the image is scaled up enough. Formally, for a scale factor `s`, an image of size `H × W` becomes `(sH) × (sW)`, and for every new pixel at destination coordinate `(x, y)` we first map it back to a (generally fractional) source coordinate `(x/s, y/s)`, then decide how to combine the source pixels near that fractional coordinate.

2.2 Nearest-neighbor interpolation

The simplest possible answer: round the fractional source coordinate to the nearest integer and copy that pixel's value directly. No new colors are ever introduced — every output pixel's value already existed somewhere in the input. This makes nearest-neighbor essentially free to compute, but because many output pixels end up copying the exact same source pixel, straight edges become staircases and the result looks visibly "blocky," especially at higher scale factors.

2.3 Bilinear interpolation

Instead of snapping to a single source pixel, bilinear interpolation looks at the **4 nearest source pixels** surrounding the fractional coordinate — the 2×2 neighborhood — and computes a weighted average, where the weight given to each of the four pixels is linear in how close the fractional coordinate is to it along the x-axis and the y-axis independently ("bi-linear" = linear in both directions, applied sequentially: first interpolate horizontally between the top two neighbors and between the bottom two, then interpolate vertically between those two results). This produces smooth gradients and removes the staircase artifacts of nearest-neighbor, at the cost of slightly blurring sharp edges, since edge pixels get blended with their neighbors.

2.4 Bicubic interpolation and overshoot

Bicubic interpolation extends the same idea to a **4×4 neighborhood** (16 source pixels) and fits a piecewise cubic polynomial through them instead of a straight line. A cubic curve can bend, which lets it approximate how natural image intensity actually varies near an edge more closely than a straight line can, typically producing sharper results than bilinear with less visible blur.

The tradeoff is a well-known artifact called **overshoot** or **ringing**: because a cubic curve can swing above or below the values it is interpolating between, the interpolated output can contain values slightly outside the original intensity range. Near a high-contrast edge this shows up as a faint halo. The figure below

demonstrates this on a purely synthetic 1-D step edge (8 samples, jumping from 0.15 to 0.85), upsampled 8× with each classic method using this project's own `upsampling.classic_upsample()` function:

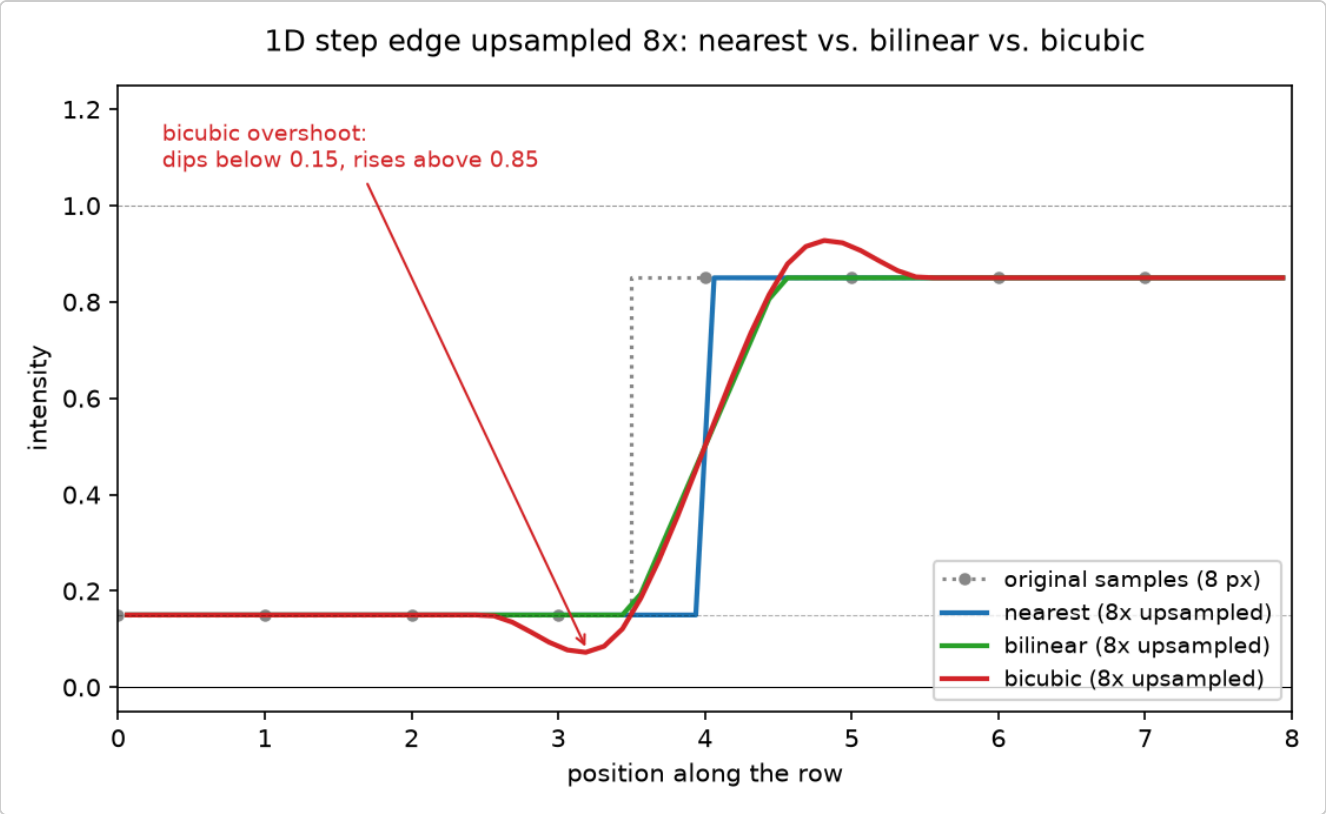


Figure 1 — Nearest-neighbor snaps to a hard step; bilinear ramps smoothly but never leaves the [0.15, 0.85] range; bicubic overshoots to roughly 0.07 and 0.93 before settling, visible as a small dip/bump right at the transition.

This is exactly why `image_utils.tensor_to_pil()` in this project clamps every result to `[0, 1]` before converting it back to a displayable 8-bit image — without that clamp, out-of-range float values would wrap around when cast to `uint8`, producing corrupted colors instead of a subtle halo.

2.5 Area interpolation

PyTorch's `area` mode implements the same idea as OpenCV's `INTER_AREA`: each output pixel is the average of the source pixels that fall within the area it covers. This is the mathematically correct way to *shrink* an image without aliasing, because every output pixel is a proper average of the source region it replaces. When used to *enlarge* an image, however, the "area" each output pixel covers in the source becomes smaller than a single source pixel, so the method degenerates toward nearest-neighbor-like behavior. It is included in this app deliberately as a pedagogical counter-example: a method well-suited to one problem (downsampling) does not automatically transfer to the opposite problem (upsampling).

2.6 `align_corners` and `antialias`

These two flags are exposed in the app for bilinear and bicubic only, because `torch.nn.functional.interpolate` only supports them for those two modes.

- **`align_corners`** changes how the source and destination pixel grids are aligned when computing each output pixel's fractional source coordinate. With it `False` (the default, and generally recommended), the grids are aligned by pixel *area* — the outer edges of the outer pixels line up. With it `True`, the *centers* of the corner pixels are forced to coincide exactly instead. The visual difference is subtle but shows up as a slight shift, most noticeable near the image borders.
- **`antialias`** applies a low-pass filter before resampling. It is primarily meant to reduce aliasing when *downsampling*; when upsampling, as in this app, its effect is a mild extra smoothing of the result.

2.7 Deep-learning super-resolution (EDSR, MSRN, DRLN)

Every method above is purely mathematical: it only ever recombines information already present in the source image. A **learned** super-resolution model instead uses a convolutional neural network, trained on many pairs of (low-resolution, high-resolution) images, to learn what plausible fine detail *usually* looks like — and then synthesizes new detail accordingly when given an unseen low-resolution input.

This app offers three such architectures, all via the open-source `super-image` package, which loads pretrained weights for each from the Hugging Face Hub. Every architecture ships in two flavors: a faster "-BAM" variant and a slower, higher-quality full variant.

- **EDSR** (Enhanced Deep residual networks for Super-Resolution, `eugenesiow/edsr-base` / `eugenesiow/edsr`) is a stack of **residual blocks** — each one a convolution, a ReLU activation, and a second convolution, with the block's own input added back to its output via a skip connection — operating at the original, low-resolution spatial size. Only at the very end of the network is the feature map actually upsampled (via sub-pixel convolution / pixel shuffling) to the target resolution. Doing the heavy computation at low resolution and upsampling only once at the end is both more efficient and lets the residual blocks focus purely on refining detail, rather than also having to process a needlessly large spatial grid throughout the whole network.
- **MSRN** (Multi-Scale Residual Network, `eugenesiow/msrn-bam` / `eugenesiow/msrn`) replaces EDSR's single-kernel-size residual blocks with **multi-scale residual blocks**: each block runs convolutions at multiple kernel sizes in parallel and combines their outputs, letting the network capture both fine and coarse detail at every stage instead of relying purely on network depth to do so.
- **DRLN** (Densely Residual Laplacian Network, `eugenesiow/drln-bam` / `eugenesiow/drln`) chains residual blocks more **densely** — each block receives the concatenated outputs of all preceding blocks, not just the previous one — and adds a **Laplacian attention** module that lets the network weight different frequency bands of detail differently, aimed specifically at recovering high-frequency texture.

Because these networks were trained specifically to reconstruct realistic high-resolution images, their output can look noticeably sharper and more detailed than any classic method at the same scale factor. But each is fundamentally making a *learned guess*, not a mathematical guarantee — it can hallucinate plausible-looking detail that was not actually present in the original scene. Like bicubic, their raw output is not guaranteed to stay within `[0, 1]`, so the same clamping step in `tensor_to_pil()` applies to all three. This project confirmed that experimentally: running the EDSR-base model on a scale-2× test image produced pixel values ranging as low as `-0.35` and as high as `1.22` before clamping.

2.8 Side-by-side comparison

The figure below was generated directly from this project's own code (`upsampling.classic_upsample` and `upsampling.sr_upsample`) applied to the bundled test-chart sample image at 4× scale, cropped to the concentric-circle quadrant to highlight ringing and staircasing, using the fast "-BAM"/"-base" variant of each learned model:

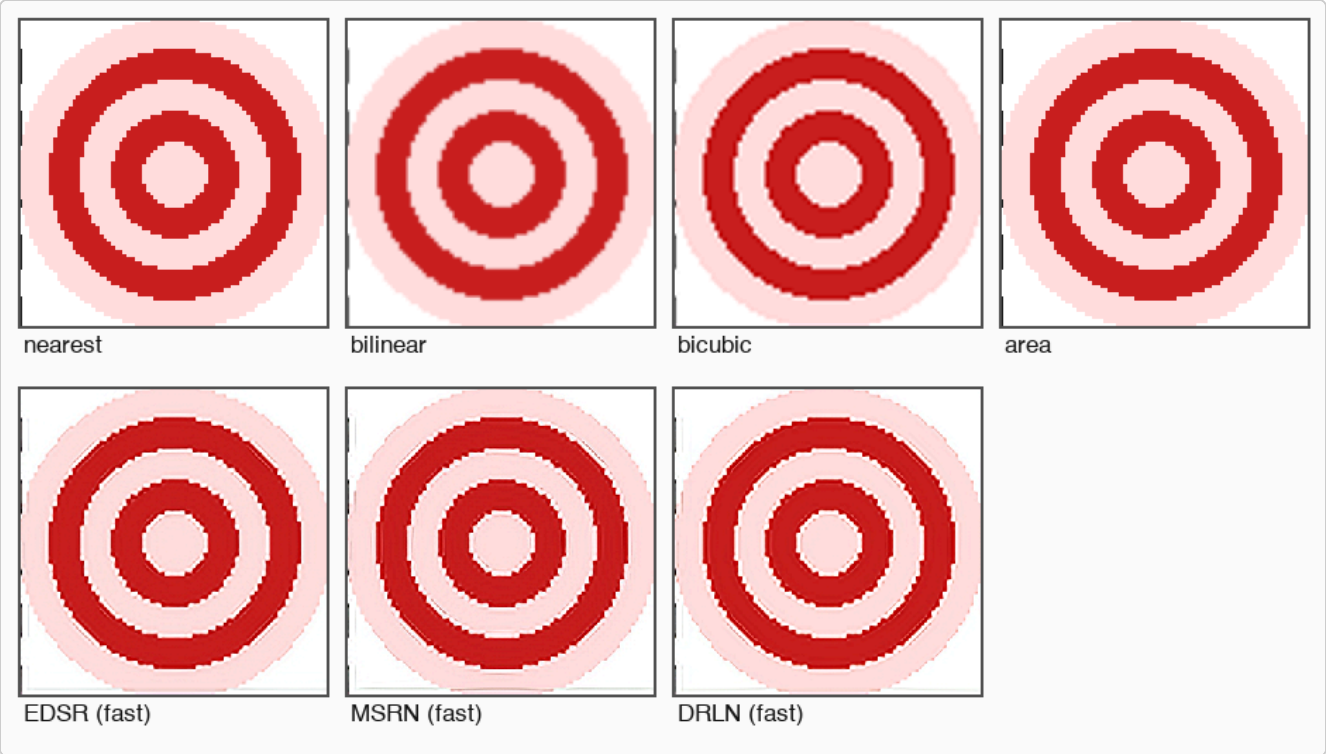


Figure 2 — Nearest and area both show hard, jagged (staircased) rings. Bilinear and bicubic are visually similar at this crop, both noticeably smoother than nearest/area, with bicubic edges slightly crisper. All three pretrained models (EDSR, MSRN, DRLN) produce noticeably smoother, more refined circular edges than any classic method, effectively having "learned" what a clean circle boundary should look like — with only subtle differences between the three at this crop, since all three are seeing the same simple synthetic pattern rather than the varied photographic textures they were actually trained to specialize on.

3. Application Walkthrough

On launch, the app loads a synthetic sample image (a checkerboard, concentric circles, a gradient, and rendered text — chosen specifically to exercise different failure modes of each method) and defaults to bicubic interpolation at 2× scale. The user can instead click **Browse...** to load any local image file, or **Load Sample 2** for a second synthetic image (soft, blurred, overlapping color blobs, which stresses the methods differently than the sharp test chart).

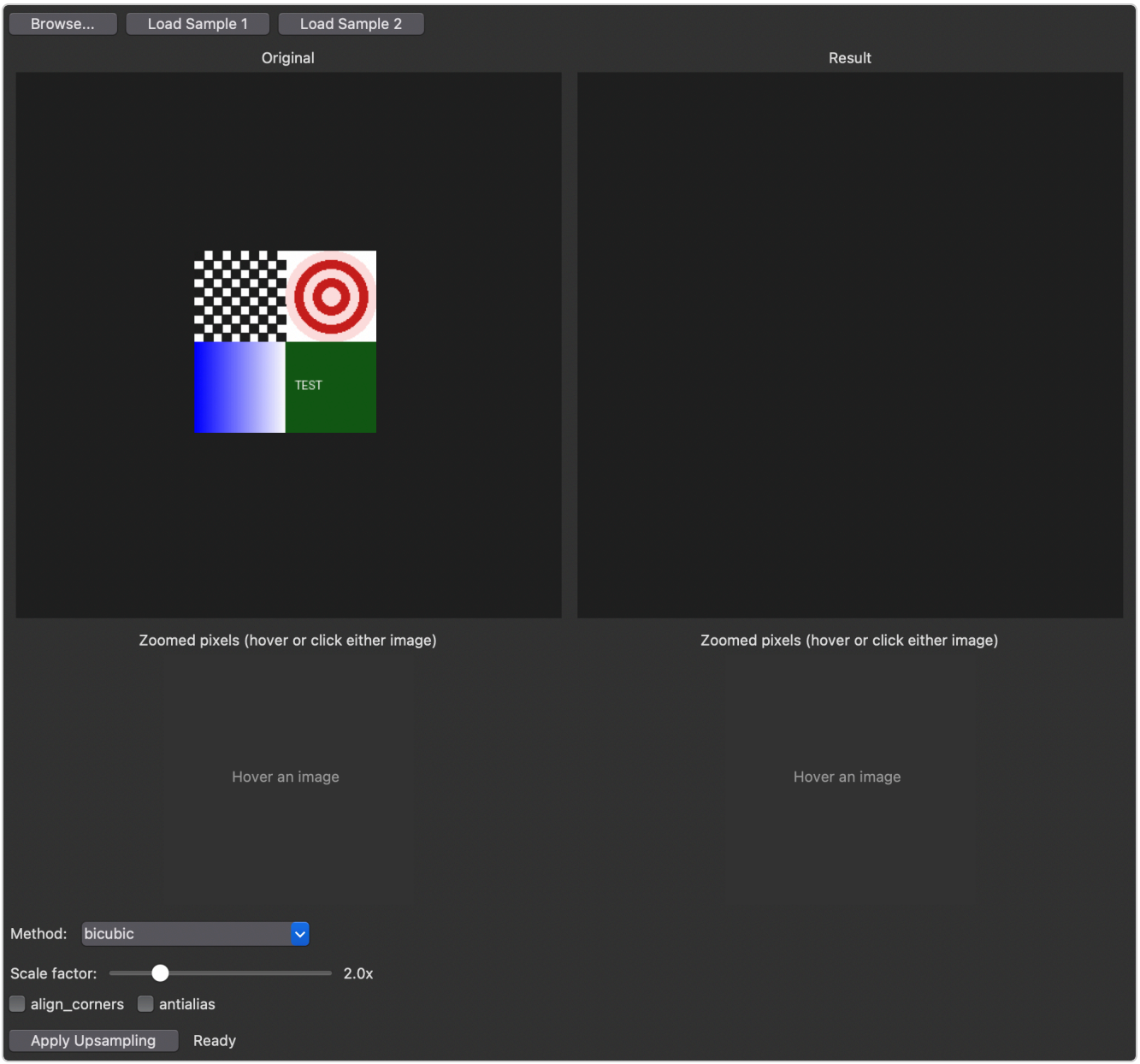


Figure 3 — Default application state: the Original pane on the left, an empty Result pane on the right awaiting the first Apply, the method dropdown with its classic-mode parameters (scale factor, align_corners, antialias), and the two "Zoomed pixels" magnifier canvases below each main pane.

Selecting **Pretrained SR** from the method dropdown swaps the parameter panel to a model selector — six variants across the three architectures (EDSR-base/EDSR, MSRN-BAM/MSRN, DRLN-BAM/DRLN) — and a scale selector restricted to {2, 3, 4}, the only scales with published pretrained weights for all three, in contrast to the classic methods' continuous scale slider. Clicking **Apply Upsampling** always runs the actual computation on a background thread, regardless of method, so the window never freezes — this matters most for these models, whose first use of a given variant also has to download its weights over the network.

The **pixel magnifier** is the app's tool for close inspection: hovering, clicking, or dragging over *either* the Original or the Result image opens a zoomed, pixel-grid view of the corresponding physical area in *both* panes simultaneously, with a yellow marker box drawn on each main image showing exactly what region is being sampled. Because the Result image can be a different resolution than the Original (e.g. 2× larger), the app converts the hovered coordinate through the actual resulting scale factor so that both magnifiers always show the same real-world patch of the picture — not the same pixel offset.

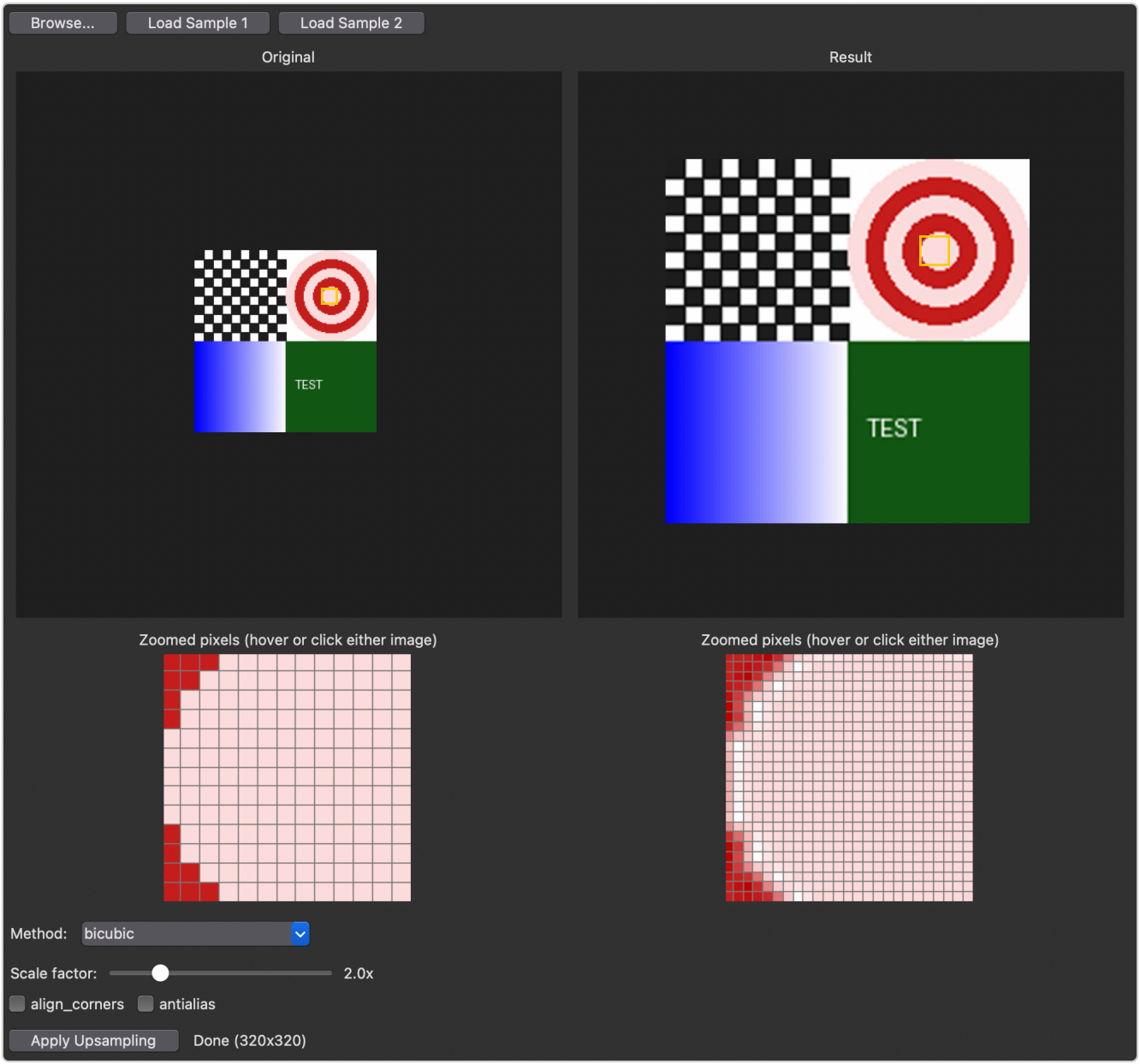


Figure 4 — The magnifier in use after applying bicubic 2× upsampling, hovering over the concentric-circle quadrant. Both zoomed views show the same physical ring boundary: blocky, sharply-stepped pixels in the 160×160 original versus a smoother transition in the 320×320 bicubic result. The yellow boxes on the main images mark the corresponding sampled regions.

4. Code Architecture

The project is deliberately flat — five small modules, no package hierarchy — split along one clear line: **GUI code never touches pixel data directly**, and **processing code never imports Tkinter**.

File	Responsibility
`main.py`	Entry point — creates the Tk root window and the `App`.
`gui.py`	All Tkinter widgets, layout, event handling, and the background-thread glue that keeps the UI responsive.
`upsampling.py`	The actual upsampling implementations — pure functions over PyTorch tensors, no Tkinter imports.
`image_utils.py`	Conversions between PIL images, PyTorch tensors, and Tkinter-displayable images, plus preview scaling and the pixel-magnifier crop/zoom logic.
`sample_images.py`	Generates two synthetic images in memory with PIL — no bundled files, no licensing concerns.

4.1 Data flow

A single request to upsample an image flows through the modules in one direction: `gui.py` hands a PIL image to `image_utils.pil_to_tensor()`, the resulting tensor goes to a function in `upsampling.py`, and the output tensor comes back through `image_utils.tensor_to_pil()` before `gui.py` displays it. Neither `upsampling.py` nor `image_utils.py` knows anything about widgets, threads, or events — they could be imported and used from a completely different interface (a CLI, a Jupyter notebook, a web backend) without change.

4.2 Threading model

Tkinter's event loop runs on the main thread; running the upsampling computation there would freeze the window for the duration — barely noticeable for classic interpolation, but very noticeable for the SR model, especially on its first run when it must also download weights over the network. `gui.py`'s `_on_apply()` always starts a `threading.Thread` running `_worker()`, which does all the actual tensor work and pushes either `("ok", result_image)` or `("error", message)` onto a `queue.Queue`. The main thread polls that queue every 100 ms via `self.after(100, self._poll_result)` — a standard Tkinter pattern for safely getting data back from a worker thread without touching any widget from outside the main thread.

4.3 The pixel-magnifier coordinate mapping

The trickiest piece of arithmetic in the app is keeping the magnifier's two views in sync despite the Original and Result images having different sizes and being displayed as downscaled previews inside fixed-size canvases. `gui.py` stores, for each pane, a small view dictionary recording the full-resolution image, the on-screen preview's pixel size, and the scale factor between them. On every mouse-move event, `_on_hover()`:

1. Converts the raw canvas (x, y) into a coordinate relative to the preview image's top-left corner (accounting for centering within the fixed-size canvas).
2. Divides by that pane's preview scale to get a coordinate in the *hovered* image's actual full-resolution pixel space.
3. If the Result pane was hovered, converts that coordinate *back* into Original-image space by dividing by the actual width/height ratio between the two images — not the requested scale factor, since real output dimensions can round slightly differently.
4. Calls `_update_magnifiers(ox, oy)` with that single, Original-space coordinate, which then re-derives the corresponding Result-space coordinate by multiplying by the same ratio, so both magnifier crops and both marker boxes are always computed from one shared source of truth.

`image_utils.magnify()` then does the actual pixel-level work: it crops a `(2×radius+1)`-pixel square centered on the given coordinate — clamping the *center*, not the crop box, so the returned patch is always full-size even near an image edge — resizes it up to a fixed display size with nearest-neighbor resampling (so individual pixels appear as flat colored blocks rather than being smoothed away), and overlays a thin grid once each source pixel maps to a large enough block on screen to make the grid useful.

> **Verification note:** because synthetic OS-level mouse-click simulation proved unreliable for automated testing on this machine, the magnifier's coordinate mapping was instead verified by calling `App._on_hover()` directly against a live `App` instance with fabricated event objects, checking that: hovering before any result exists leaves the Result magnifier in its placeholder state; hovering after Apply populates both magnifiers and both marker boxes; hovering the Result canvas at the geometrically corresponding point produces the same pair of magnifier images as hovering the equivalent point on the Original canvas; and leaving either canvas clears all magnifier state. All four checks passed.

5. Module-by-Module Code Listing

5.1 main.py

The entire entry point — construct the root window and the app, then hand control to Tkinter's event loop.

main.py

```
import tkinter as tk

from gui import App

if __name__ == "__main__":
    root = tk.Tk()
    root.title("Upsampling Comparison")
    App(root).pack(fill="both", expand=True)
    root.mainloop()
```

5.2 upsampling.py

Every upsampling algorithm the app can run, expressed as pure functions over PyTorch tensors. `classic_upsample()` only passes `align_corners` and `antialias` through when the mode actually supports them, since PyTorch raises an error if they are supplied for `nearest` or `area`. `SR_VARIANTS` maps each display name to an `(architecture, repo_id)` pair, and `_model_class()` resolves the architecture string to the matching `super_image` class (`EdsrModel`, `MsrnModel`, or `DrlnModel`) — adding a further architecture is then a matter of extending that one mapping and the `SR_VARIANTS` dictionary, not touching the GUI at all. `get_sr_model()` caches each loaded model by `(repo_id, scale)` so switching back to a previously-used variant is instant, and the `super_image` import is deferred into `_model_class()` so the base application never pays the cost of importing OpenCV/Hugging Face Hub unless the user actually selects the SR method.

upsampling.py

```
"""Upsampling implementations: classic torch interpolation and pretrained
super-resolution. No GUI imports here -- kept independent of Tkinter so it
can be tested/reused on its own.
"""

import torch
import torch.nn.functional as F

CLASSIC_MODES = ["nearest", "bilinear", "bicubic", "area"]

# Each variant maps to (architecture, repo_id). The architecture selects
```

```
# which super_image model class knows how to load that repo's weights.
SR_VARIANTS = {
    "EDSR-base (fast)": ("edsr", "eugenesiow/edsr-base"),
    "EDSR (quality)": ("edsr", "eugenesiow/edsr"),
    "MSRN-BAM (fast)": ("msrn", "eugenesiow/msrn-bam"),
    "MSRN (quality)": ("msrn", "eugenesiow/msrn"),
    "DRLN-BAM (fast)": ("drln", "eugenesiow/drln-bam"),
    "DRLN (quality)": ("drln", "eugenesiow/drln"),
}
SR_SCALES = [2, 3, 4]

_sr_model_cache = {}  # (repo_id, scale) -> loaded model


def classic_upsample(tensor, mode, scale_factor, align_corners, antialias):
    """Upsample a (1,3,H,W) float tensor in [0,1] using F.interpolate."""
    kwargs = {}
    if mode in ("bilinear", "bicubic"):
        kwargs["align_corners"] = align_corners
        kwargs["antialias"] = antialias
    with torch.no_grad():
        return F.interpolate(tensor, scale_factor=scale_factor, mode=mode, **kwargs)


def _model_class(architecture):
    # Deferred: avoid importing opencv/huggingface_hub on startup.
    from super_image import DrlnModel, EdsrModel, MsrnModel

    return {"edsr": EdsrModel, "msrn": MsrnModel, "drln": DrlnModel}[architecture]


def get_sr_model(architecture, repo_id, scale):
    key = (repo_id, scale)
    if key not in _sr_model_cache:
        model = _model_class(architecture).from_pretrained(repo_id, scale=scale)
        model.eval()
        _sr_model_cache[key] = model
    return _sr_model_cache[key]


def sr_upsample(tensor, architecture, repo_id, scale):
    """Run a pretrained super-resolution model on a (1,3,H,W) tensor in [0,1]."""
    model = get_sr_model(architecture, repo_id, scale)
    with torch.no_grad():
        return model(tensor)
```

5.3 image_utils.py

All conversions between PIL images, PyTorch tensors, and Tkinter-displayable images live here, along with the magnifier's crop/zoom logic described in section 4.3. `tensor_to_pil()`'s clamp to `[0, 1]` is the single most theory-relevant line in the whole codebase — it is what makes bicubic's overshoot (section 2.4) and the SR models' out-of-range outputs (section 2.7) render as a subtle visual softening instead of corrupted, wrapped-around colors.

image_utils.py

```
"""PIL <-> torch tensor <-> Tkinter PhotoImage conversions, plus preview
scaling. No GUI event handling here -- just data conversion helpers.
"""

import torch
from PIL import Image, ImageDraw, ImageTk
import torchvision.transforms.functional as TF


def load_image_from_path(path):
    """Load an image file as RGB. Raises on bad/corrupt files -- caller
    should catch (OSError, ValueError) and show an error dialog."""
    return Image.open(path).convert("RGB")


def guard_image_size(image, max_dim=1600):
    """Downscale in place (returns a possibly-new image) if either
    dimension exceeds max_dim. Returns (image, was_resized)."""
    if max(image.size) > max_dim:
        image = image.copy()
        image.thumbnail((max_dim, max_dim), Image.LANCZOS)
        return image, True
    return image, False


def pil_to_tensor(image):
    """RGB PIL Image -> (1,3,H,W) float32 tensor in [0,1]."""
    return TF.to_tensor(image).unsqueeze(0)


def tensor_to_pil(tensor):
    """(1,3,H,W) float tensor -> RGB PIL Image.

    Clamps to [0,1] before converting: bicubic overshoot/ringing and SR
    model outputs can exceed that range, which would otherwise wrap
    around when cast to uint8.
    """
    tensor = tensor.squeeze(0).clamp(0, 1)
    array = tensor.mul(255).round().to("cpu", dtype=torch.uint8)
    array = array.permute(1, 2, 0).numpy()
    return Image.fromarray(array, "RGB")


def make_preview(image, max_size=(480, 480)):
    """Display-only downscaled copy. Actual upsampling always runs on the
    full-resolution source image, independent of this."""
    preview = image.copy()
    preview.thumbnail(max_size, Image.LANCZOS)
    return preview


def magnify(image, center_x, center_y, radius, target_size):
```

```
    """Crop a (2*radius+1)-square pixel patch centered on (center_x,
    center_y) and scale it up to target_size with nearest-neighbor
    resampling, so individual source pixels are visible as flat blocks.
    A thin grid is overlaid on top of the pixel boundaries when each
    source pixel maps to a large enough block to make it useful.

    The center is clamped so the returned patch is always the full
    requested size (never distorted by clipping at the image border).
    """
    radius = max(1, int(round(radius)))
    box = radius * 2 + 1
    width, height = image.size

    if width <= box:
        left, right = 0, width
    else:
        cx = max(radius, min(center_x, width - radius - 1))
        left = int(round(cx - radius))
        right = left + box

    if height <= box:
        top, bottom = 0, height
    else:
        cy = max(radius, min(center_y, height - radius - 1))
        top = int(round(cy - radius))
        bottom = top + box

    patch = image.crop((left, top, right, bottom))
    patch = patch.resize((target_size, target_size), Image.NEAREST)

    cols, rows = right - left, bottom - top
    cell_w, cell_h = target_size / cols, target_size / rows
    if cell_w >= 6 and cell_h >= 6:
        draw = ImageDraw.Draw(patch)
        for i in range(1, cols):
            x = round(i * cell_w)
            draw.line([(x, 0), (x, target_size)], fill=(120, 120, 120))
        for j in range(1, rows):
            y = round(j * cell_h)
            draw.line([(0, y), (target_size, y)], fill=(120, 120, 120))

    return patch


def to_photoimage(image):
    """Wrap a PIL Image for Tkinter display. Caller must keep a strong
    reference (e.g. self._photo = ...) or Tkinter will blank the canvas
    once the PhotoImage is garbage-collected."""
    return ImageTk.PhotoImage(image)
```

5.4 sample_images.py

Two images generated entirely in memory with PIL primitives, so the app has something useful to show immediately with no external files or licensing concerns. The test chart deliberately packs four different

visual challenges into one small image: a checkerboard (aliasing), concentric circles (ringing/staircasing), a gradient (smooth tonal transitions), and rendered text (edge sharpness).

sample_images.py

```
"""Synthetic sample images generated in-memory with PIL -- no bundled
files, no licensing concerns. Deliberately small (~160px) so upsampled
differences between methods are visually obvious and SR inference stays
fast on CPU.
"""

from PIL import Image, ImageDraw, ImageFilter


def make_test_chart(size=160):
    """Checkerboard, concentric circles, a gradient band, and text --
    good for showing aliasing, ringing, and edge-sharpness differences
    between upsampling methods."""
    image = Image.new("RGB", (size, size), "white")
    draw = ImageDraw.Draw(image)
    half = size // 2

    # Top-left: checkerboard (aliasing / moire demo)
    cell = max(size // 20, 2)
    for y in range(0, half, cell):
        for x in range(0, half, cell):
            if ((x // cell) + (y // cell)) % 2 == 0:
                draw.rectangle([x, y, x + cell, y + cell], fill=(30, 30, 30))

    # Top-right: concentric circles (ringing / staircasing demo)
    cx, cy = half + half // 2, half // 2
    for r in range(half // 2, 0, -8):
        color = (200, 30, 30) if (r // 8) % 2 == 0 else (255, 220, 220)
        draw.ellipse([cx - r, cy - r, cx + r, cy + r], fill=color)

    # Bottom-left: smooth horizontal gradient
    for x in range(half):
        shade = int(255 * x / half)
        draw.line([(x, half), (x, size)], fill=(shade, shade, 255))

    # Bottom-right: text (edge-sharpness demo)
    draw.rectangle([half, half, size, size], fill=(20, 90, 20))
    draw.text((half + 8, half + half // 2 - 8), "TEST", fill="white")

    return image


def make_soft_blobs(size=160):
    """Overlapping colored ellipses with Gaussian blur -- mimics soft
    photographic gradients, stressing methods differently than the sharp
    test chart."""
    image = Image.new("RGB", (size, size), (250, 245, 235))
    draw = ImageDraw.Draw(image)
```

```
    blobs = [
        (0.25, 0.30, 0.28, (220, 90, 90)),
        (0.65, 0.35, 0.32, (90, 140, 220)),
        (0.45, 0.70, 0.30, (110, 200, 130)),
        (0.75, 0.75, 0.20, (230, 200, 90)),
    ]
    for cx, cy, r, color in blobs:
        x, y, rad = cx * size, cy * size, r * size
        draw.ellipse([x - rad, y - rad, x + rad, y + rad], fill=color)

    return image.filter(ImageFilter.GaussianBlur(radius=2))
```

5.5 gui.py

The complete Tkinter interface: widget layout, the method/parameter panel that swaps between classic and SR controls, the background-thread Apply flow (section 4.2), and the pixel-magnifier event handlers (section 4.3).

gui.py

```
"""Tkinter GUI: layout, widgets, event handlers, and threading glue.
Delegates all actual image processing to upsampling.py / image_utils.py.
"""

import queue
import threading
import tkinter as tk
from tkinter import ttk, filedialog, messagebox

import image_utils
import sample_images
import upsampling

PANE_SIZE = (480, 480)

MAGNIFIER_DISPLAY = 220  # size (px) of the zoomed-pixel canvases
MAGNIFIER_RADIUS = 6     # half-width, in *original*-image pixels, of the sampled area


class App(ttk.Frame):
    def __init__(self, master):
        super().__init__(master)
        self.source_image = None  # full-resolution PIL image
        self.result_image = None  # full-resolution upsampled PIL image (after Apply)
        self._original_photo = None
        self._result_photo = None
        self._original_zoom_photo = None
        self._result_zoom_photo = None
        self._original_view = None  # {"image", "preview_size", "scale"} for coord mapping
        self._result_view = None
        self._original_box_id = None
        self._result_box_id = None
```

```
        self._queue = queue.Queue()

        self._build_widgets()
        self.load_sample(1)

    # ---- layout ------------------------------------------------------

    def _build_widgets(self):
        top_bar = ttk.Frame(self)
        top_bar.pack(side="top", fill="x", padx=8, pady=6)
        ttk.Button(top_bar, text="Browse...", command=self._on_browse).pack(side="left")
        ttk.Button(top_bar, text="Load Sample 1",
                   command=lambda: self.load_sample(1)).pack(side="left", padx=(6, 0))
        ttk.Button(top_bar, text="Load Sample 2",
                   command=lambda: self.load_sample(2)).pack(side="left", padx=(6, 0))

        panes = ttk.Frame(self)
        panes.pack(side="top", fill="both", expand=True, padx=8)

        original_col = ttk.Frame(panes)
        original_col.pack(side="left", padx=4)
        ttk.Label(original_col, text="Original").pack()
        self.original_canvas = tk.Canvas(original_col, width=PANE_SIZE[0],
                                         height=PANE_SIZE[1], background="#222")
        self.original_canvas.pack()
        ttk.Label(original_col, text="Zoomed pixels (hover or click either image)").pack(pady=(6,
        self.original_zoom_canvas = tk.Canvas(original_col, width=MAGNIFIER_DISPLAY,
                                              height=MAGNIFIER_DISPLAY, background="#333")
        self.original_zoom_canvas.pack()

        result_col = ttk.Frame(panes)
        result_col.pack(side="left", padx=4)
        ttk.Label(result_col, text="Result").pack()
        self.result_canvas = tk.Canvas(result_col, width=PANE_SIZE[0],
                                       height=PANE_SIZE[1], background="#222")
        self.result_canvas.pack()
        ttk.Label(result_col, text="Zoomed pixels (hover or click either image)").pack(pady=(6, 0
        self.result_zoom_canvas = tk.Canvas(result_col, width=MAGNIFIER_DISPLAY,
                                            height=MAGNIFIER_DISPLAY, background="#333")
        self.result_zoom_canvas.pack()

        for event in ("<Motion>", "<Button-1>", "<B1-Motion>"):
            self.original_canvas.bind(event, lambda e: self._on_hover("original", e))
            self.result_canvas.bind(event, lambda e: self._on_hover("result", e))
        self.original_canvas.bind("<Leave>", lambda e: self._on_leave())
        self.result_canvas.bind("<Leave>", lambda e: self._on_leave())
        self._clear_zoom_canvas(self.original_zoom_canvas, "Hover an image")
        self._clear_zoom_canvas(self.result_zoom_canvas, "Hover an image")

        controls = ttk.Frame(self)
        controls.pack(side="top", fill="x", padx=8, pady=8)

        method_row = ttk.Frame(controls)
        method_row.pack(side="top", fill="x")
        ttk.Label(method_row, text="Method:").pack(side="left")
        self.method_names = upsampling.CLASSIC_MODES + ["Pretrained SR"]
```

```
        self.method_var = tk.StringVar(value="bicubic")
        method_box = ttk.Combobox(method_row, textvariable=self.method_var,
                                  values=self.method_names, state="readonly")
        method_box.pack(side="left", padx=(6, 0))
        method_box.bind("<<ComboboxSelected>>", lambda e: self._on_method_change())

        self.param_frame = ttk.Frame(controls)
        self.param_frame.pack(side="top", fill="x", pady=(8, 0))

        self._build_classic_params()
        self._build_sr_params()
        self._on_method_change()

        action_row = ttk.Frame(controls)
        action_row.pack(side="top", fill="x", pady=(8, 0))
        self.apply_btn = ttk.Button(action_row, text="Apply Upsampling",
                                    command=self._on_apply)
        self.apply_btn.pack(side="left")
        self.status_var = tk.StringVar(value="Ready")
        ttk.Label(action_row, textvariable=self.status_var).pack(side="left", padx=(10, 0))
        self.progress = ttk.Progressbar(action_row, mode="indeterminate", length=150)

    def _build_classic_params(self):
        frame = ttk.Frame(self.param_frame)
        self.classic_frame = frame

        scale_row = ttk.Frame(frame)
        scale_row.pack(side="top", fill="x")
        ttk.Label(scale_row, text="Scale factor:").pack(side="left")
        self.scale_var = tk.DoubleVar(value=2.0)
        self.scale_label_var = tk.StringVar(value="2.0x")
        ttk.Scale(scale_row, from_=1.0, to=6.0, variable=self.scale_var,
                  orient="horizontal", length=200,
                  command=self._on_scale_drag).pack(side="left", padx=(6, 6))
        ttk.Label(scale_row, textvariable=self.scale_label_var).pack(side="left")

        opts_row = ttk.Frame(frame)
        opts_row.pack(side="top", fill="x", pady=(4, 0))
        self.align_corners_var = tk.BooleanVar(value=False)
        self.antialias_var = tk.BooleanVar(value=False)
        self.align_corners_check = ttk.Checkbutton(
            opts_row, text="align_corners", variable=self.align_corners_var)
        self.antialias_check = ttk.Checkbutton(
            opts_row, text="antialias", variable=self.antialias_var)
        self.align_corners_check.pack(side="left")
        self.antialias_check.pack(side="left", padx=(10, 0))

    def _build_sr_params(self):
        frame = ttk.Frame(self.param_frame)
        self.sr_frame = frame

        variant_row = ttk.Frame(frame)
        variant_row.pack(side="top", fill="x")
        ttk.Label(variant_row, text="Model:").pack(side="left")
        self.sr_variant_var = tk.StringVar(value="EDSR-base (fast)")
        ttk.Combobox(variant_row, textvariable=self.sr_variant_var,
```

```
                    values=list(upsampling.SR_VARIANTS.keys()),
                    state="readonly").pack(side="left", padx=(6, 0))

        scale_row = ttk.Frame(frame)
        scale_row.pack(side="top", fill="x", pady=(4, 0))
        ttk.Label(scale_row, text="Scale:").pack(side="left")
        self.sr_scale_var = tk.IntVar(value=2)
        ttk.Combobox(scale_row, textvariable=self.sr_scale_var,
                    values=upsampling.SR_SCALES, state="readonly",
                    width=5).pack(side="left", padx=(6, 0))

    # ---- event handlers -----------------------------------------

    def _on_scale_drag(self, value):
        self.scale_label_var.set(f"{float(value):.1f}x")

    def _on_method_change(self):
        is_sr = self.method_var.get() == "Pretrained SR"
        self.classic_frame.pack_forget()
        self.sr_frame.pack_forget()
        if is_sr:
            self.sr_frame.pack(side="top", fill="x")
        else:
            self.classic_frame.pack(side="top", fill="x")
            is_area_or_nearest = self.method_var.get() in ("nearest", "area")
            state = "disabled" if is_area_or_nearest else "normal"
            self.align_corners_check.config(state=state)
            self.antialias_check.config(state=state)

    def _on_browse(self):
        path = filedialog.askopenfilename(
            filetypes=[("Images", "*.png *.jpg *.jpeg *.bmp *.gif *.webp"),
                       ("All files", "*.*")])
        if not path:
            return
        try:
            image = image_utils.load_image_from_path(path)
        except (OSError, ValueError):
            messagebox.showerror("Load failed", f"Could not open image:\n{path}")
            return
        self._set_source_image(image)

    def load_sample(self, which):
        image = sample_images.make_test_chart() if which == 1 else sample_images.make_soft_blobs(
        self._set_source_image(image)

    def _set_source_image(self, image):
        image, was_resized = image_utils.guard_image_size(image)
        self.source_image = image
        if was_resized:
            self.status_var.set("Image was large; downscaled to fit.")
        preview = image_utils.make_preview(image, PANE_SIZE)
        self._original_photo = image_utils.to_photoimage(preview)
        self.original_canvas.delete("all")
        self.original_canvas.create_image(PANE_SIZE[0] // 2, PANE_SIZE[1] // 2,
                                          image=self._original_photo, anchor="center")
```

```
        self._original_view = self._make_view(image, preview)
        self._original_box_id = None

        # Previous result no longer corresponds to the new source image.
        self.result_image = None
        self._result_view = None
        self._result_box_id = None
        self.result_canvas.delete("all")
        self._on_leave()

    def _collect_params(self):
        method = self.method_var.get()
        if method == "Pretrained SR":
            architecture, repo_id = upsampling.SR_VARIANTS[self.sr_variant_var.get()]
            return {
                "method": "sr",
                "architecture": architecture,
                "repo_id": repo_id,
                "scale": self.sr_scale_var.get(),
            }
        return {
            "method": "classic",
            "mode": method,
            "scale_factor": self.scale_var.get(),
            "align_corners": self.align_corners_var.get(),
            "antialias": self.antialias_var.get(),
        }

    def _on_apply(self):
        if self.source_image is None:
            return
        params = self._collect_params()
        self.apply_btn.config(state="disabled")
        self.progress.pack(side="left", padx=(10, 0))
        self.progress.start(10)
        if params["method"] == "sr":
            self.status_var.set("Running super-resolution (first use downloads the model)...")
        else:
            self.status_var.set("Processing...")

        source_image = self.source_image
        threading.Thread(target=self._worker, args=(source_image, params), daemon=True).start()
        self.after(100, self._poll_result)

    def _worker(self, source_image, params):
        try:
            tensor = image_utils.pil_to_tensor(source_image)
            if params["method"] == "sr":
                out = upsampling.sr_upsample(
                    tensor, params["architecture"], params["repo_id"], params["scale"])
            else:
                out = upsampling.classic_upsample(
                    tensor, params["mode"], params["scale_factor"],
                    params["align_corners"], params["antialias"])
            result_image = image_utils.tensor_to_pil(out)
            self._queue.put(("ok", result_image))
```

```
        except Exception as exc:
            self._queue.put(("error", str(exc)))

    def _poll_result(self):
        try:
            status, payload = self._queue.get_nowait()
        except queue.Empty:
            self.after(100, self._poll_result)
            return

        self.progress.stop()
        self.progress.pack_forget()
        self.apply_btn.config(state="normal")

        if status == "ok":
            self.status_var.set(f"Done ({payload.width}x{payload.height})")
            self._display_result(payload)
        else:
            self.status_var.set("Error")
            messagebox.showerror(
                "Upsampling failed",
                f"{payload}\n\nIf you were using the pretrained SR model, this may mean "
                "the model weights could not be downloaded (check your internet connection).")

    def _display_result(self, image):
        self.result_image = image
        preview = image_utils.make_preview(image, PANE_SIZE)
        self._result_photo = image_utils.to_photoimage(preview)
        self.result_canvas.delete("all")
        self.result_canvas.create_image(PANE_SIZE[0] // 2, PANE_SIZE[1] // 2,
                                        image=self._result_photo, anchor="center")
        self._result_view = self._make_view(image, preview)
        self._result_box_id = None

    # ---- magnifying glass ----------------------------------------------

    @staticmethod
    def _make_view(full_image, preview):
        return {
            "image": full_image,
            "preview_size": preview.size,
            "scale": preview.width / full_image.width,
        }

    def _clear_zoom_canvas(self, canvas, message):
        canvas.delete("all")
        canvas.create_text(MAGNIFIER_DISPLAY // 2, MAGNIFIER_DISPLAY // 2,
                           text=message, fill="#999")

    def _on_leave(self):
        self._set_box(self.original_canvas, "_original_box_id", None)
        self._set_box(self.result_canvas, "_result_box_id", None)
        self._clear_zoom_canvas(self.original_zoom_canvas, "Hover an image")
        self._clear_zoom_canvas(self.result_zoom_canvas, "Hover an image")

    def _on_hover(self, pane, event):
```

```
        view = self._original_view if pane == "original" else self._result_view
        if view is None:
            return

        pw, ph = view["preview_size"]
        x0 = PANE_SIZE[0] / 2 - pw / 2
        y0 = PANE_SIZE[1] / 2 - ph / 2
        px, py = event.x - x0, event.y - y0
        if not (0 <= px < pw and 0 <= py < ph):
            self._on_leave()
            return

        fx, fy = px / view["scale"], py / view["scale"]
        if pane == "original":
            ox, oy = fx, fy
        else:
            rx = self.source_image.width / self.result_image.width
            ry = self.source_image.height / self.result_image.height
            ox, oy = fx * rx, fy * ry

        self._update_magnifiers(ox, oy)

    def _update_magnifiers(self, ox, oy):
        orig_patch = image_utils.magnify(self.source_image, ox, oy,
                                         MAGNIFIER_RADIUS, MAGNIFIER_DISPLAY)
        self._original_zoom_photo = image_utils.to_photoimage(orig_patch)
        self.original_zoom_canvas.delete("all")
        self.original_zoom_canvas.create_image(0, 0, image=self._original_zoom_photo,
                                               anchor="nw")
        self._set_box(self.original_canvas, "_original_box_id",
                      self._preview_rect(self._original_view, ox, oy, MAGNIFIER_RADIUS))

        if self.result_image is None:
            self._clear_zoom_canvas(self.result_zoom_canvas, "Click Apply first")
            self._set_box(self.result_canvas, "_result_box_id", None)
            return

        rx = self.result_image.width / self.source_image.width
        ry = self.result_image.height / self.source_image.height
        result_patch = image_utils.magnify(self.result_image, ox * rx, oy * ry,
                                           MAGNIFIER_RADIUS * max(rx, ry), MAGNIFIER_DISPLAY)
        self._result_zoom_photo = image_utils.to_photoimage(result_patch)
        self.result_zoom_canvas.delete("all")
        self.result_zoom_canvas.create_image(0, 0, image=self._result_zoom_photo, anchor="nw")
        self._set_box(self.result_canvas, "_result_box_id",
                      self._preview_rect(self._result_view, ox * rx, oy * ry,
                                         MAGNIFIER_RADIUS * max(rx, ry)))

    @staticmethod
    def _preview_rect(view, cx, cy, radius):
        pw, ph = view["preview_size"]
        scale = view["scale"]
        x0 = PANE_SIZE[0] / 2 - pw / 2
        y0 = PANE_SIZE[1] / 2 - ph / 2
        return (x0 + (cx - radius) * scale, y0 + (cy - radius) * scale,
                x0 + (cx + radius + 1) * scale, y0 + (cy + radius + 1) * scale)
```

```
    def _set_box(self, canvas, attr, coords):
        box_id = getattr(self, attr)
        if coords is None:
            if box_id is not None:
                canvas.delete(box_id)
                setattr(self, attr, None)
            return
        if box_id is None:
            box_id = canvas.create_rectangle(*coords, outline="#ffcc00", width=2)
            setattr(self, attr, box_id)
        else:
            canvas.coords(box_id, *coords)
```

6. Design Decisions Summary

Decision	Rationale
Explicit "Apply" button rather than live-updating on every parameter change	SR inference (and its first-run download) can take seconds; a single interaction model for both classic and SR methods avoids inconsistent UX and keeps one code path.
Always run Apply on a background thread, even for fast classic methods	Keeps one uniform, always-correct code path rather than special-casing "classic is fast enough to run inline."
`super-image`'s pretrained models (EDSR, MSRN, DRLN) instead of training a model from scratch	Real pretrained weights with a small, well-defined dependency footprint, rather than needing training data and compute this project doesn't have. A shared `(architecture, repo_id)` mapping keeps adding a new architecture to a one-line change in `upsampling.py`.
Clamp every result to [0, 1] before display	Both bicubic and the SR model can produce out-of-range float values (sections 2.4, 2.7); without clamping, casting to 8-bit color wraps around into corrupted pixels.
Synthetic in-memory sample images instead of bundled photo files	Avoids any image licensing question and lets the samples be purpose-built to exercise specific artifacts (aliasing, ringing, edge sharpness).
Clamp the magnifier's sample *center*, not its crop box, near image edges	Keeps the zoomed patch always the same physical size, avoiding visual distortion when hovering near a border.